

Lab8 mmap

Task：修改xv6代码，实现mmap功能

- mmap：把“文件的一段内容”映射到“进程的一段虚拟地址空间”里，让程序可以像读写普通内存一样读写文件。

普通文件 I/O 是这样：

```
c
char buf[4096];
read(fd, buf, 4096);
buf[0] = 'A';
write(fd, buf, 4096);
```

而 mmap 是这样：

```
c
char *p = mmap(..., fd, ...);
p[0] = 'A';
munmap(p, length);
```

- 也就是说，mmap 把显式的 read/write 变成了普通的内存访问。

mmap是如何实现的，它做了什么？

- 从操作系统角度，mmap 并不是马上把整个文件读进内存。

mmap做的是：

1. 在进程虚拟地址空间里找一段空区域
2. 记录这段虚拟地址对应某个文件的某个范围
3. 等程序真的访问这段地址时，再通过 page fault 读入对应文件页
4. 如果是 MAP_SHARED，修改最终可以写回文件

所以mmap的核心是：虚拟地址空间 + 文件 + page fault + 懒加载

mmap的意义

- 让文件访问更自然。
 - 对程序员来说，文件内容像数组一样：
- 按需加载，节省内存和时间。
 - 如果映射一个 1GB 文件，mmap 不会立刻读 1GB。
 - 程序访问第一页，就只加载第一页。
 - 访问第 1000 页，就只加载第 1000 页。
- 可以减少不必要的数据复制。
 - 普通 read 通常是：磁盘 / buffer cache -> 内核缓冲区 -> 用户 buffer
 - mmap page fault：磁盘/buffer cache -> 新分配的物理页 -> 建立用户虚拟地址映射
- 可以实现进程间共享内存。
 - 如果两个进程 MAP_SHARED 映射同一个文件，理论上它们可以看到同一份内容。

Implementation

1. 添加新的系统调用mmap和munmap（参考Lab 2）
2. 按照实验要求，，为每个struct proc增加一个长度为16的vma表

```
// 虚拟内存区域，记录一次 mmap 映射
struct vma {
    int valid;           // 该 VMA 表项是否正在使用
    uint64 addr;         // 映射区域的起始虚拟地址
    int length;          // 映射区域长度，单位为字节
    int prot;            // 映射权限，如 PROT_READ / PROT_WRITE
    int flags;           // 映射类型，如 MAP_SHARED / MAP_PRIVATE
    int fd;              // mmap 时传入的原始文件描述符
    int offset;          // 映射内容在文件中的起始偏移
    struct file *f;      // 对应的内核文件对象，防止 close(fd) 后失效
};
```

```
#define NVMA 16
// Per-process state
struct proc {
    ... 省略 ...
    char name[16];
    struct vma vmas[NVMA];
};
```

核心思想是：mmap 不是马上读文件，而是先保存一条“虚拟地址到文件”的映射记录。

Implementation

3. 实现sys_mmap功能 (kernel/sysfile.c)

- 解析参数
- 做权限和地址检查
- 找一个空 VMA 表项并填进去

注意：这里不分配物理页，不读文件，不建立页表映射。

Implementation

4. 在 usertrap 里处理 mmap page fault(kernel/trap.c):

- 用户第一次访问 mmap 返回的地址时, 因为还没有物理页, 会触发 page fault。
- 在usertrap中处理:
 - 读取 fault address, 也就是 r_stval()
 - 判断它是否落在某个 VMA 区域里
 - 如果是, 分配一页物理内存
 - 从文件对应 offset 读一页内容到物理页
 - 根据 VMA 的 prot 设置页表权限
 - 调用 mappages 建立 va -> pa 映射

Implementation

5. 实现sys_munmap ()功能 (kernel/sysfile.c)

- 根据 addr 找到对应 VMA
- 如果是 MAP_SHARED, 把内容写回文件
- 调用 uvmunmap 删除页表映射并释放物理页
- 根据取消的是头部、尾部还是整个区域, 更新 VMA
- 如果整个 VMA 都取消了, 调用 fileclose(f), 并把 valid 置 0

实验保证 munmap 只会取消开头、结尾或整个区域, 不会在中间挖洞。

因为 mmap 是懒加载, 有些页可能没被访问过、没有真实映射, 所以 uvmunmap 可能需要改成遇到未映射页时跳过。

Implementation

6. 修改exit函数, 对所有没有释放掉的vma调用munmap
 - 如果是 MAP_SHARED, 写回文件
 - 释放页表映射和物理页
 - fileclose(f)
 - 清空 VMA
7. 修改fork函数, 使得子进程拥有和父进程相同的映射区域
 - 复制父进程的 VMA 表到子进程
 - 对每个有效 VMA 的 struct file * 调用 filedup

Improvement

- 目标：减少物理页使用，重用 buffer cache 中的数据。
- 当所读取的映射区域在buffer cache中已经存在时，不要申请新的物理地址，而是使用已经存在的buffer cache
 - 更新BSIZE大小为PGSIZE
 - 给buffer cache加引用计数或 pin 计数，防止其被替换掉
 - page fault 时用 bread，而不是 kalloc + readi
 - VMA 或页表旁边要记住这个 va 对应哪个 buf，将来 munmap 时，你要知道该 unpin 哪个 buffer。

- 本次实验是30分，希望大家认真完成。